

AI-Powered Role-Based Candidate Screening System

A Retrieval-Augmented Generation (RAG) interview platform

Technical Documentation

SECTION 1

Executive Overview

This project is an **AI-powered, role-based candidate screening system**. A candidate uploads a resume and selects a target role; the system then conducts a structured technical interview in which the questions are **not predefined** — they are generated dynamically from three inputs: the candidate's resume, the selected role, and a role-specific knowledge base (an authoritative textbook).

The intelligence at its core is a **Retrieval-Augmented Generation (RAG)** pipeline. Rather than asking a language model to invent questions from memory, the system first *retrieves* the most relevant passages from the textbook and then *generates* questions grounded in those passages. Every question is traceable back to the exact source material that produced it, and the interview adapts to the candidate's answers as it progresses.

The full lifecycle handled by the system:

- **Candidate entry** — upload a resume (PDF or text) and pick a role.
- **Resume processing** — parse the resume and extract skills, technologies, domains and seniority.
- **Context construction** — turn resume + role into targeted retrieval queries.
- **Knowledge retrieval (RAG)** — fetch the most relevant, grounded chunks from the role's knowledge base.
- **Question generation** — produce interview questions grounded in the retrieved context and tailored to the candidate.

- **Interactive interview** — the candidate answers in the UI; each next question adapts to previous answers.
- **Response handling & storage** — persist every question, answer, and its grounding trace.
- **Final output** — a structured summary with strengths, gaps, a score and a recommendation.

SECTION 2

How the System Works — End-to-End Flow

The pipeline is a single directed flow. The guiding principle is **Context -> Question -> Answer -> Storage**: nothing is generated without grounding context, and nothing is discarded without being stored.

```
[1] Candidate Entry upload resume (PDF/text) + select a role
[2] Resume Processing parse -> extract skills / technologies / domains
[3] Context Construction build resume + role driven retrieval queries
[4] Knowledge Retrieval embed queries -> top-k chunks from the book (RAG)
[5] Question Generation grounded, resume-aware question (one at a time)
[6] Interactive Interview candidate answers -> next question adapts
[7] Response Storage persist question + answer + grounding trace
[8] Final Output structured summary: strengths, gaps, score

(steps 4->7 repeat for each question until the interview ends)
```

A short worked example: a candidate whose resume emphasises *NLP* and *PyTorch* triggers retrieval of passages on neural networks, regularization and generalization; the generator then asks a question such as “how would you reduce overfitting in a text-classification network?” — grounded in the retrieved passages and matched to the candidate’s stated seniority. A candidate strong in reinforcement learning would be routed to different chapters entirely.

SECTION 3

Technology Stack — What It Uses

Frontend	React 18 + Vite. Inter typeface, warm minimal design system, light/dark theme.
Backend	Python 3.11 + FastAPI (async web framework) with Uvicorn.
Text generation	Groq — Llama 3.3 70B (fast, generous free tier). Routed via a provider abstraction.
Embeddings	Google Gemini — gemini-embedding-001 (3072-dim vectors).
Vector database	ChromaDB — persistent, embedded, cosine similarity search.
Relational storage	SQLite via SQLAlchemy (SQLAlchemy + Pydantic). Swappable to PostgreSQL.
Resume parsing	pypdf for PDF text extraction; LLM-based structured profiling.

Config	pydantic-settings — all values from environment / .env, no hard-coded secrets.
--------	--

Why a hybrid of two AI providers? Groq serves open-weight LLMs with a fast, generous free tier and strong reasoning — ideal for question generation. Groq, however, has *no embeddings API*, so embeddings use Gemini, whose embedding free tier is ample. A one-line setting (LLM_PROVIDER) selects the generation provider; everything routes through a single module, so switching is trivial.

SECTION 4

System Architecture

The system is a modular, layered application — not a single script. Each layer has one responsibility, which keeps the code testable and easy to extend.

routers/	HTTP transport only: parse requests, map errors to status codes.
services/interview_service.py	Orchestration: the single source of truth for how a session advances.
services/ (RAG)	One module per stage: parsing, chunking, embeddings, vector store, retrieval, generation, summary.
services/llm.py	Provider dispatcher for generation (Groq Gemini).
models.py + database.py	Persistence with SQLAlchemy over SQLite.
config.py + roles.py	Environment-driven configuration and the role registry.

Request & data flow

```

React (Vite) --HTTP/JSON--> FastAPI routers
|
v
interview_service (orchestrator)
|
+-----+-----+-----+
| | | |
resume_parser retrieval question_generator summary_service
| | |
vector_store llm (Groq) llm (Groq)
(ChromaDB)
|
gemini_client (embeddings) SQLite (SQLModel)

```

SECTION 5

Core Concepts Behind It

Retrieval-Augmented Generation (RAG)

Instead of trusting an LLM's parametric memory, RAG supplies the model with relevant external text at inference time. This grounds outputs in a trusted source, reduces hallucination, and makes answers auditable. Here it ensures every interview question comes from the assigned textbook.

Embeddings & semantic search

An embedding maps a piece of text to a high-dimensional vector so that semantically similar texts sit close together. Retrieval embeds the query and finds the nearest chunk vectors by cosine similarity — matching on *meaning*, not keywords.

Chunking

A 400-page book cannot be embedded as one vector. It is split into overlapping, boundary-aware chunks so each is a focused, self-contained retrieval unit. Overlap preserves ideas that straddle a boundary; chunk size trades retrieval precision against context completeness.

Vector database

ChromaDB stores the chunk vectors and performs fast nearest-neighbour search. It runs embedded (no server) and persists to disk, so ingestion is a one-time step and retrieval is instant.

Resume-driven / dynamic retrieval

Queries are synthesised from the candidate's own resume signals paired with the role, so retrieval — and therefore the whole interview — is personalised rather than generic.

Adaptive, grounded generation

Questions are generated one at a time, each conditioned on the retrieved context, the questions already asked, and the candidate's most recent answer — enabling follow-ups and difficulty calibration.

Traceability

Each question stores the exact chunks that grounded it (source + similarity + snippet). The chain Context -> Question -> Answer -> Storage is fully reconstructable and surfaced in the UI.

SECTION 6

The RAG Pipeline in Detail

6.1 Knowledge ingestion (offline)

Run once via `scripts/ingest.py`. It reads each role's documents, chunks them, embeds the chunks with Gemini, and stores them in ChromaDB tagged with the role. Because Gemini's free embedding tier counts one request per document and caps at 100/min, ingestion uses a sliding-window rate limiter and larger chunks so a full textbook fits under the cap. The Tom Mitchell *Machine Learning* book produced **499 chunks**.

```
def chunk_text(text, *, chunk_size, chunk_overlap):
    text = _normalise(text)
    units = _split_units(text) # paragraphs -> sentences -> words
    chunks, current = [], ''
    for unit in units:
        candidate = f'{current}\n\n{unit}'.strip() if current else unit
        if len(candidate) <= chunk_size:
            current = candidate # keep packing the chunk
        else:
            chunks.append(current)
            tail = current[-chunk_overlap:] # carry overlap forward
            current = f'{tail}\n\n{unit}'.strip()
        ...
    return [Chunk(text=c, index=i) for i, c in enumerate(chunks)]
```

6.2 Retrieval mechanism

At interview time, the system builds several queries from the resume profile and role, embeds each, queries ChromaDB (filtered to the role), and fuses the results — de-duplicating chunks and keeping the best similarity score.

```
def build_queries(*, profile, role_id):
    role = get_role(role_id)
    signals = dedup(profile.domains + profile.skills + profile.technologies)
    queries = [f'{role.label} concepts related to {s}' for s in signals[:5]]
    queries.append(f'Core {role.label} fundamentals: {role.description}')
    return dedup(queries)

def retrieve_context(*, profile, role_id, top_k):
    best = {}
    for q in build_queries(profile=profile, role_id=role_id):
        emb = gemini_client.embed_query(q)
        for chunk in vector_store.query(query_embedding=emb,
                                       role=role_id, top_k=top_k):
            key = (chunk.source, chunk.chunk_index)
            if key not in best or chunk.score > best[key].score:
                best[key] = chunk # fuse, keep highest score
    return sorted(best.values(), key=lambda c: c.score, reverse=True)[:top_k]
```

6.3 Question generation

The generator prompts the LLM with the role, the resume profile, the retrieved context, the questions already asked, and the last answer. It returns structured JSON: the question, a topic, a difficulty, a one-line rationale, and the indices of the context chunks it used — which are then persisted for traceability.

```
def generate_question(*, role_id, profile, context,
                    asked_questions, last_question, last_answer):
    prompt = SYSTEM_PROMPT.format(role=..., profile=..., context=context,
                                   asked=asked_questions,
                                   followup=(last_question, last_answer))
    data = llm.generate_json(prompt, temperature=0.6)
    return GeneratedQuestion(question=data['question'], topic=data['topic'],
                             difficulty=data['difficulty'],
                             rationale=data['rationale'],
                             context_indices=data['context_indices'])
```

SECTION 7

Data Model & Persistence

Two SQLAlchemy tables capture the full record. A session owns an ordered list of question/answer turns, and each turn keeps its own grounding trace — so the provenance of every question survives in the database, not just in memory.

InterviewSession

- candidate name, role, full resume text, structured resume_profile
- status (in_progress / completed), and the final summary (insights JSON)

QAPair (one per turn)

- topic, difficulty, rationale — why the question was asked
- context_chunks — the grounding trace (source, similarity, snippet)
- question, answer, timestamps

SECTION 8

The Main Function That Ties Everything Together

The orchestrator in `services/interview_service.py` is the heart of the backend. Two functions drive the entire lifecycle; the web layer only calls into them.

Starting an interview

```
def start_interview(db, *, role_id, candidate_name, resume_text):
    profile = resume_parser.build_profile(resume_text, role_id) # extract
    session = InterviewSession(role=role_id, resume_profile=profile, ...)
    db.add(session); db.commit()
    _generate_and_store_question(db, session) # first grounded question
    return session
```

Generating & storing the next question (retrieval + generation + persistence)

```
def _generate_and_store_question(db, session):
    profile = ResumeProfile(**session.resume_profile)
    context = retrieval.retrieve_context(profile=profile, role_id=session.role)
    existing = _ordered_questions(db, session)
    last = existing[-1] if existing else None
    gen = question_generator.generate_question(
        role_id=session.role, profile=profile, context=context,
        asked_questions=[q.question for q in existing],
        last_question=last.question if last else None,
        last_answer=last.answer if last else None)
    used = gen.context_indices or list(range(min(3, len(context))))
    qa = QAPair(session_id=session.id, order_index=len(existing),
        topic=gen.topic, difficulty=gen.difficulty,
        rationale=gen.rationale, question=gen.question,
        context_chunks=[trace(context[i]) for i in used])
    db.add(qa); db.commit()
```

Advancing the interview

```
def submit_answer(db, *, session_id, answer):
    session = _get_session(db, session_id)
    pending = _pending_question(db, session) # the open question
    pending.answer = answer; db.commit()
    if _answered_count(db, session) >= QUESTIONS_PER_INTERVIEW:
        _finalise(db, session) # generate summary/insights
    else:
        _generate_and_store_question(db, session) # adaptive next question
    return session
```

This is where all the pieces meet: **resume parsing** feeds **retrieval**, which feeds **generation**, whose output (with its grounding trace) is **persisted**, and the answer loop repeats until the session is **summarised**.

SECTION 9

Key Design Decisions & Trade-offs

- **Question-at-a-time generation** costs one LLM call per turn but unlocks adaptivity and keeps each question grounded in freshly-retrieved context.
- **Hybrid providers (Groq + Gemini)** use each provider where it is strongest — Groq for fast, generous generation; Gemini for embeddings.
- **Robustness over brittleness:** resume profiling and summary generation have deterministic fallbacks, and JSON parsing has a salvage path, so one model hiccup never breaks the flow.
- **Traceability as a feature:** grounding chunks are stored and shown in the UI, not used and discarded.
- **Designing around real constraints:** ingestion is rate-limited and uses larger chunks to fit a whole textbook under the free embedding cap.
- **Roles are data, not code:** adding a role is one registry entry plus a folder of documents.
- **Config via environment:** no secrets in code; SQLite->PostgreSQL is a connection-string change.

SECTION 10

How to Run

Backend

```
cd backend
python -m venv .venv && .venv\Scripts\activate
pip install -r requirements.txt
copy .env.example .env # set GROQ_API_KEY and GOOGLE_API_KEY
python -m scripts.ingest --role ai_ml_engineer # one-time ingestion
uvicorn app.main:app --reload --port 8000
```

Frontend

```
cd frontend
npm install
npm run dev # http://localhost:5173
```

Open the app, read the landing page, upload a resume, choose the AI/ML Engineer role, and the system runs a grounded, adaptive interview ending in a structured assessment.